

UniVerify: Benchmarki warstwy 2 w rejestrze dyplomów opartym na Ethereum

Porównanie Sepolia, Arbitrum, Base oraz zkSync

Nazar Shcherbyna
Praca magisterska

Pomiar wykonano: 2026-05-19T08:04:28.281Z

Spis treści

1	Wprowadzenie	2
2	Metodologia	2
2.1	Pomiary	2
2.2	Parametry próby	2
3	Wyniki: koszt issueBatch	2
4	Wyniki: koszt revokeFromBatch	3
5	Wyniki: latencja	3
6	Wyniki: analizy wrażliwości	4
6.1	Scenariusze basefee L1	4
6.2	Wrażliwość na kurs ETH/USD	4
6.3	Wrażliwość na ceny gazu sekwensera L2	4
7	Dyskusja	5
7.1	Ograniczenia pomiaru	5
7.2	Reprodukowalność	6

1 Wprowadzenie

Niniejszy rozdział porównuje koszt uruchomienia kontraktu `DiplomaRegistry` na czterech sieciach Ethereum: warstwie pierwszej (Sepolia, jako punkt odniesienia) oraz trzech rozwiązaniach warstwy drugiej reprezentujących różne rodziny (Arbitrum Nitro, Base jako przedstawiciel OP-stack, zkSync Era jako rollup z dowodami zwięzłości).

2 Metodologia

2.1 Pomiary

Każda sieć mierzona jest na sieci testowej: *zużycie gazu* i *opłata za dane L1* są wielkościami rzeczywistymi, natomiast koszty w USD są modelowane z historycznych wartości `baseFeePerGas` warstwy pierwszej z ostatnich 90 dni (percentyle p10/p50/p90).

2.2 Parametry próby

Dla `issueBatch` użyto rozmiarów paczek {1, 10, 100, 1 000, 10 000}. Dla `revokeFromBatch` zebrano 30 próbek przeciwko paczce seed o rozmiarze 40. Dla `statusWithProof` zebrano 100 próbek czasu odpowiedzi RPC.

3 Wyniki: koszt `issueBatch`

Tabela 1: Tabela 1a: Zużycie gazu `issueBatch` według sieci i rozmiaru paczki. Wartości deterministyczne (pojedyncza próbka per (sieć, rozmiar)).

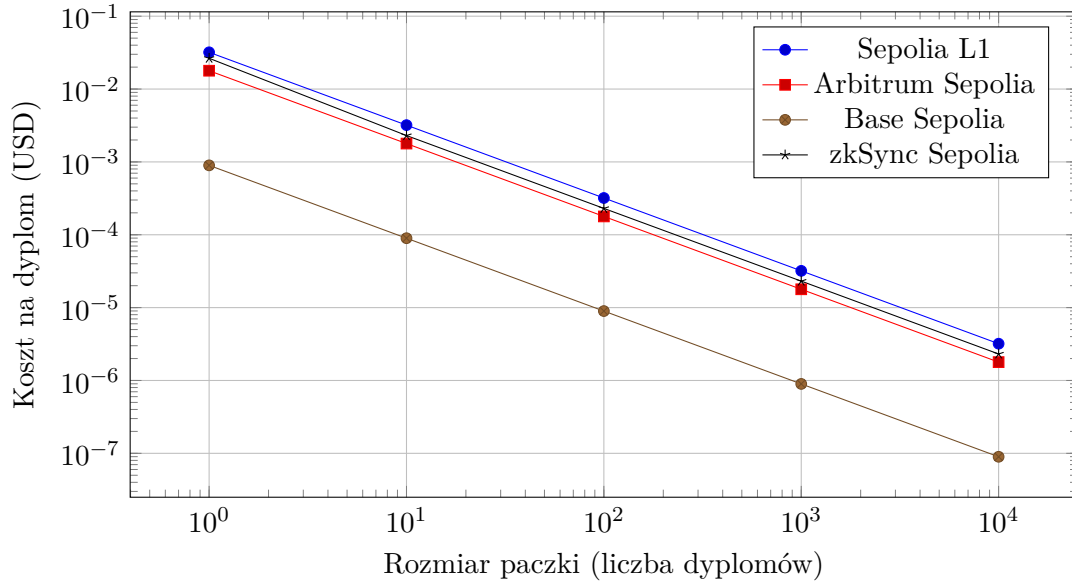
Sieć	$n=1$	$n=10$	$n=100$	$n=1000$	$n=10000$
sepolia	51012	51024	51024	51024	51024
arbitrumSepolia	51012	51012	51000	51012	51024
baseSepolia	51012	51012	51012	51012	51012
zksyncSepolia	151171	131468	131462	132010	132010

Tabela 2: Tabela 1b: Koszt `issueBatch` w USD (scenariusz p50, ETH/USD=3500). Pełna zmienność scenariuszy p10/p50/p90 dla $n=1000$ na Rys. 4.

Sieć	$n=1$	$n=10$	$n=100$	$n=1000$	$n=10000$
sepolia	0.031961	0.031968	0.031968	0.031968	0.031968
arbitrumSepolia	0.017857	0.017857	0.017852	0.017857	0.017861
baseSepolia	0.000897	0.000897	0.000897	0.000897	0.000897
zksyncSepolia	0.026455	0.023007	0.023006	0.023102	0.023102

Tabela 3: Tabela 2: Koszt na dyplom (amortyzacja paczki) w USD, scenariusz p50.

Sieć	$n=1$	$n=10$	$n=100$	$n=1000$	$n=10000$
sepolia	0.031960695	0.003196821	0.000319682	0.000031968	0.000003197
arbitrumSepolia	0.017856626	0.001785663	0.000178524	0.000017857	0.000001786
baseSepolia	0.000896818	0.000089682	0.000008968	0.000000897	0.000000090
zksyncSepolia	0.026454925	0.002300690	0.000230058	0.000023102	0.000002310



Rysunek 1: Rys. 1: Koszt na dyplom w funkcji rozmiaru paczki (skala log-log, scenariusz p50).

4 Wyniki: koszt revokeFromBatch

Tabela 4: Tabela 3: Koszt `revokeFromBatch` (pojedyncza transakcja, mediana).

chain	median_gas	median_lldata_wei	usd_p50
sepolia	59798	0	0.037465
arbitrumSepolia	60221	8342502000	0.021089
baseSepolia	59796	53792586	0.001066
zksyncSepolia	223176	0	0.039056

5 Wyniki: latencja

Tabela 5: Tabela 4: Latencja odczytu `statusWithProof`.

chain	p50_ms	p95_ms	max_observed_ms
sepolia	39.080022000242025	61.91715699993074	279.1554379998706
arbitrumSepolia	38.742031999863684	50.90663200011477	127.85965100023896
baseSepolia	204.46466600010172	207.53652499988675	308.2396650002338
zksyncSepolia	172.1015240000561	206.19782000000123	486.5620189999754

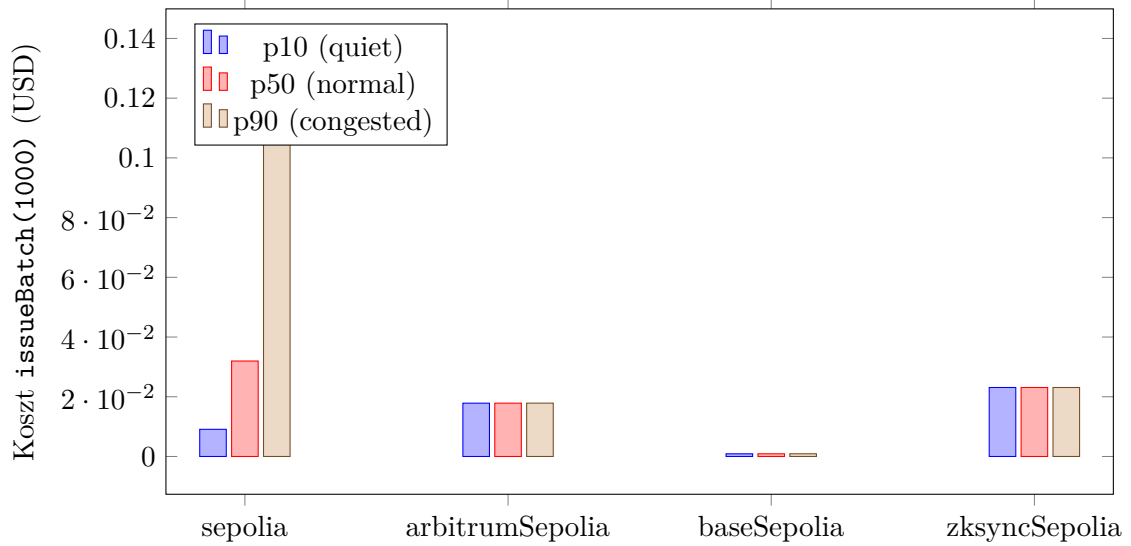
Tabela 6: Tabela 5: Czas odbioru paragonu transakcji zapisu po stronie klienta (ms). Kolumny `issue` pochodzą z $N=30$ powtórzeń `issueBatch(1000)` w każdym runie i są agregowane przez wszystkie trzy niezależne runy.

Sieć	revoke p_{50}	revoke p_{95}	issue p_{50}	issue p_{95}	issue σ	N
sepolia	12499	33086	12503	48897	11923	90
arbitrumSepolia	491	727	407	644	126	90
baseSepolia	1538	2145	1494	2391	678	90
zksyncSepolia	2254	6181	2114	6068	1954	90

Mierzona wartość to czas między wywołaniem `sendTransaction` a otrzymaniem paragonu przez klienta RPC. Przy interwale odpytywania około 4 s (domyślnym w bibliotece `viem`) wynik jest górnym ograniczeniem faktycznego czasu inkluzji w bloku, nie jego dokładnym pomiarem; rozkład wartości na sieci Sepolia jest wyraźnie skupiony wokół wielokrotności czasu bloku (≈ 12 s), co potwierdza powyższą interpretację.

6 Wyniki: analizy wrażliwości

6.1 Scenariusze basefee L1



Rysunek 2: Rys. 4: Koszt `issueBatch(1000)` przy scenariuszach basefee p10/p50/p90.

6.2 Wrażliwość na kurs ETH/USD

Koszt w USD jest liniową funkcją kursu ETH/USD: $C(r) = C_{3500} \cdot r/3500$, gdzie C_{3500} to wartość bazowa zaraportowana przy snapshotcie z dnia 2026-05-13 ($r = 3500$). Tabela 6 pokazuje koszt `issueBatch(1000)` przy scenariuszu basefee p50 dla trzech zakotwiczonych poziomów rynkowych.

Tabela 7: Tabela 6: Wrażliwość kosztu `issueBatch(1000)` na kurs ETH/USD (basefee p50).

chain	usd_2000	usd_3500	usd_5000
sepolia	0.018268	0.031968	0.045669
arbitrumSepolia	0.010204	0.017857	0.025509
baseSepolia	0.000512	0.000897	0.001281
zksyncSepolia	0.013201	0.023102	0.033002

6.3 Wrażliwość na ceny gazu sekwensera L2

Cena gazu ustalana przez operatora sekwensera L2 jest niezależna od L1 basefee. Model kosztu rozkłada się addytywnie:

$$C_{\text{total}} = C_{\text{exec}} + C_{\text{L1 data}},$$

gdzie $C_{\text{exec}} = \text{gasUsed} \cdot p_{\text{L2}}$ skaluje się liniowo z ceną sekwensera p_{L2} , natomiast $C_{\text{L1 data}}$ pochodzi z blob basefee na L1 i pozostaje stała. Tabela 7 pokazuje koszt `issueBatch(1000)` przy trzech mnożnikach względem snapshotu cen w `packages/benchmarks/src/config.ts` (Arbitrum 0.1 gwei, Base 0.005 gwei, zkSync 0.05 gwei). Sepolia pominięta — jako sieć L1 nie ma ceny sekwensera.

Tabela 8: Tabela 7: Wrażliwość kosztu `issueBatch(1000)` na cenę gazu sekwensera L2 (ETH/USD=3500, basefee p50; mnożniki względem snapshotu z `config.ts`).

chain	usd_05x	usd_10x	usd_20x
arbitrumSepolia	0.008930	0.017857	0.035711
baseSepolia	0.000450	0.000897	0.001790
zksyncSepolia	0.011551	0.023102	0.046204

7 Dyskusja

Wszystkie pomiary opisane w niniejszym rozdziale wykonano przez jednego dostawcę RPC w jednej lokalizacji POP, z $N=30$ powtórzeniami dla każdego pomiaru opóźnienia oraz losowym wyborem liści przy revoke. Wyniki to średnia $\pm\sigma$ z trzech niezależnych runów. Wszystkie parametry konfiguracyjne (ziarno losowości, dostawca, region, kurs ETH/USD) są zapisane w pliku `12-benchmarks/data/meta.json`.

7.1 Ograniczenia pomiaru

Poniżej zebrano metodologiczne ograniczenia, których czytelnik powinien być świadomy interpretując wyniki. Cztery z pierwotnie zidentyfikowanych ograniczeń zostały wyeliminowane w toku rewizji eksperymentu (jednolity dostawca RPC, $N=30$ powtórzeń, losowy wybór liści do revoke, trzy niezależne runy z raportowaniem σ); pozostałe sześć omówiono poniżej.

Założenie kompresji Brotli dla Arbitrum. Model kosztu zapisu danych na L1 dla Arbitrum przyjmuje współczynnik kompresji Brotli równy 1.0, ponieważ dane Merkle (hashe pseudolosowe) nie kompresują się znacząco. Realny aplikacyjny calldata o strukturze rzadkiej kompresuje się 2–4-krotnie, co oznacza, że nasze oszacowanie kosztu L1 dla Arbitrum jest **górnym ograniczeniem**; rzeczywiste koszty produkcyjne mogą być proporcjonalnie niższe.

Niewspółmierność jednostki gazu w zkSync Era. Wartość `gasUsed` raportowana przez zkSync (131 463 dla `issueBatch` z $n=1$) łączy w jednej jednostce koszt wykonania, dane publiczne i narzut bootloadera kontekstu Account Abstraction — nie jest to ta sama jednostka, co `gasUsed` w EVM. Porównanie surowych liczb gazu między rodzinami rollupów jest **metodologicznie niepoprawne**; sensowne porównanie kosztów może odbywać się wyłącznie w USD lub ETH.

Opóźnienie inkluzji jest ograniczone przez interwał odpytywania. Pomiar czasu od submit do receipt opiera się o cykl odpytywania klienta viem (~ 4 s). Otrzymane wartości są zatem **górnym ograniczeniem czasu inkluzji w bloku**, a nie samym czasem inkluzji. Porównanie międzysieciowe jest dodatkowo zniekształcone interakcją czasu bloku z interwałem odpytywania.

Snapshot skalarów Base SystemConfig. Model kosztu Base Ecotone używa snapshotu parametrów `baseFeeScalar=2269` oraz `blobBaseFeeScalar=1055762` pobranego z bloku L1 25087416. Po zmianie tych skalarów przez operatora Base wynikowe koszty zmieniłyby się proporcjonalnie; data snapshotu jest udokumentowana w `meta.json` każdego eksportu.

Snapshot kursu ETH/USD. Wyniki w USD przeliczono kursem ETH/USD = 3500 (snapshot CoinGecko z dnia 2026-05-13). Wynik dla innego kursu można uzyskać mnożeniem przez czynnik $\frac{\text{nowy kurs}}{3500}$.

Konserwatywne ceny sekwensera L2. Ceny gazu sekwensera L2 zostały ustawione na zaokrąglone wartości powyżej żywego snapshotu (Arbitrum 0.1 gwei vs 0.02 żywe; Base 0.005 vs 0.006; zkSync 0.05 vs 0.0453), aby pochłoniąć typowe wahania kongestii.

7.2 Reprodukowalność

Wszystkie wyniki w tym rozdziale można odtworzyć z czystego klona repozytorium UniVerify poleceniem `npm run benchmarks:all -- --runs=3` po wypełnieniu pliku `.env` kluczem prywatnym portfela testowego oraz adresami RPC czterech sieci testnet (Sepolia, Arbitrum Sepolia, Base Sepolia, zkSync Sepolia). Pipeline wykonuje sekwencję `deploy` → `measure` → `price` → `aggregate` → `export`, regenerując wszystkie tabele i wykresy w `docs/12-benchmarks/data/`. Każdy artefakt jest powiązany z plikiem `meta.json` zawierającym `runId`, `measuredAt`, kurs ETH/USD z datą snapshotu, ziarno losowości (`BENCH_RANDOM_SEED`), dostawcę i region RPC oraz listę łańcuchów. Plik `packages/benchmarks/README.md` dokumentuje wymagania środowiskowe i znane ograniczenia. CI repozytorium (`.github/workflows/benchmarks-drift.yml`) automatycznie regeneruje tabele z za-commitowanych próbek wyników i sygnalizuje rozjazd między źródłem a zatwierdzonymi danymi.

Wniosek. ...